Comparative Analysis of DCGAN and WGAN-GP

Aatyanth Thimma Udayakumar [1] **Vishal Patel** [1]

Abstract

Since the introduction of Generative Adversarial Networks (GANs), the image synthesis space has changed completely. More advanced GAN models, such as DCGAN and WGAN-GP, have pushed the boundaries of creating realistic images and stabilizing training. In this project, we compare the performance of DCGAN and WGAN-GP across three datasets of varying complexity: MNIST, CIFAR-10, and Anime Faces. We evaluate each model's ability to generate realistic, diverse images on qualitative and quantitative metrics (Fréchet Inception Distance and Inception Score).

1. Introduction

Generative Models have transformed the field of image generation, allowing computers to create realistic images from random noise. Specifically, with the introduction of Generative Adversarial Networks (GANs) (Goodfellow et al., 2014), we have seen these image qualities improve exponentially with the flexibility of having a separate discriminator and generator trained on some sample noise data. GANs leverage two distinct neural networks: the Generator and the Discriminator. The Generator's role is to produce the synthetic image while the Discriminator's role is to classify whether the generated image is real or fake. This adversarial training allows the generator to create more realistic images since by receiving meaningful feedback from the discriminator.

In 2017, promising developments in the field of GANs led to the creation of a variety of specialized GANs with specific purposes and use cases. Among these, the Deep Convolutional Generative Adversarial Network (DCGAN) (Radford et al., 2016) and the Wasserstein GAN with Gradient Penalty (WGAN-GP) (Gulrajani et al., 2017) are some of the most well-researched and utilized GANs for simple image generation. The DCGAN architecture is often called the standard GAN, well known for its intuitive convolutional approach for the generator and discriminator, while the WGAN-GP builds upon the DCGAN to improve the stability of training using the Wasserstein-1 distance and gradient penalties.

Although much of the research with GANs has been devoted to optimizing existing models or stabilizing training, there is limited work being done to compare the performance of these GAN models across datasets. In our project, we aim to compare the performance of the DCGAN and WGAN-GP models on 3 different datasets of increasing difficulty: The MNIST collection, the CIFAR-10 collection, and an Anime Face collection. Our primary goal in this project is to generate realistic images (based on the training dataset). We aim to assess the quality, diversity, and realism of the generated images by comparing the performance of the two GAN architectures across the three datasets. Through this experimentation, we also hope to gain insight into how GANs work with different complexities of datasets, as well as the applicability of these GANs.

2. Methods

2.1. Dataset Description

2.1.1. MNIST

The MNIST (Modified National Institute of Standards and Technology) dataset is a collection of handwritten digit images. The total dataset contains 60,000 training images and 10,000 testing images of 28x28 grayscale images. Each image represents a handwritten digit from 0-9 and has relatively low variability in the overall style and background, making each image's variance minimal, with just the actual digit being different. We chose this dataset as a baseline to be able to evaluate the GAN models due to the simplicity of the images.

To use this dataset in our code, we utilized PyTorch's built-in dataset library (Torchvision.datasets), allowing us to directly download it to our Jupyter Notebooks for further analysis. We trained both GANs on the entire MNIST dataset.

2.1.2. CIFAR-10

The CIFAR-10 (Canadian Institute for Advanced Research) dataset is a collection of 60,000, 32x32 colored images from 10 different classes: cars, birds, cats, deer, dogs, frogs, horses, ships, and trucks. As such, each class in the dataset has roughly 6,000 images. We labeled this as a medium-level dataset because the training images themselves are of low resolution and blurred, theoretically reducing the

amount of detail the models would have to capture to generate high-quality fakes. However, this dataset still contains a lot more detail than simple, black-and-white, handwritten, digits, making it a much harder challenge for the GANs than the MNIST dataset.

To use this dataset in our code, we utilized PyTorch's built-in dataset library (Torchvision.datasets), allowing us to directly download it to our Jupyter Notebooks for further analysis. We trained both GANs on a subset of about 10,000 images to limit computational overhead.

2.1.3. Anime Faces

The Anime Faces dataset is a collection of 63,000, high-resolution anime character faces. All the images in the dataset are strictly cropped to focus only on the actual faces of the characters, omitting any other artistic elements. This measure helps bring consistency between the images and ensures that the only variance between images are facial expressions, hair colors, accessories, etc. We chose this dataset to be the advanced dataset because the GANs would not only have to mimic the Anime art-style, but also reproduce the high-resolution of the real images to generate high-quality outputs with good variation. This mandates the GANs to capture significantly more subtle features/details, making this a significantly harder dataset compared to MNIST and CIFAR-10.

To use this dataset in our code, we directly imported the dataset onto our Jupyter Notebooks from Kaggle (*link to dataset*) using the kagglehub module. We trained both GANs on a subset of about 10,000 images to limit computational overhead.

2.2. Model Architecture & Loss Function

2.2.1. DCGAN

The Deep Convolutional Generative Adversarial Network (DCGAN) is composed of two deep Convolutional Neural Networks that work together: The Generator and Discriminator (Radford et al., 2016). This architecture was introduced in 2015 and is considered one of the most popular for high-quality image generation (Radford et al., 2016).

Generator Architecture:

The Generator part of DCGAN is designed to produce artificially images that resemble real images from the training dataset. The idea behind it is to map a random latent vector 'z,' sampled from a Gaussian distribution, to a high-dimensional image using a series of transposed convolution layers. Each convolutional transpose layer is paired with a 2d batch norm layer and a ReLU activation function. Therefore, the generator's main objective is to fool the discriminator into classifying the generated image as real. This is achieved by progressively up-sampling the latent vector 'z' through several layers of de-convolution, each increasing the resolution of the output image. Finally, the generator utilizes a tanh activation function in the last layer to ensure the output falls within the range of [-1, 1].

Layers

- **Input Layer:** A random latent vector size z sampled from a Gaussian Distribution is used as the input to the generator to represent random noise that the generator will use to convert into a realistic image
- **Output:** The output layer is the final layer that samples the output to the desired image size (64x64) using nc filters, where nc is the number of channels in the output image. The output is then passed through a Tanh activation function to map the pixel values to the range of [-1,1].

```
==========================================================================================
Layer (type:depth-idx)                   Output Shape              Param #
==========================================================================================
├─Sequential: 1-1                        [-1, 3, 1072, 1072]       --
|    └─ConvTranspose2d: 2-1              [-1, 512, 67, 67]         819,200
|    └─BatchNorm2d: 2-2                  [-1, 512, 67, 67]         1,024
|    └─ReLU: 2-3                         [-1, 512, 67, 67]         --
|    └─ConvTranspose2d: 2-4              [-1, 256, 134, 134]       2,097,152
|    └─BatchNorm2d: 2-5                  [-1, 256, 134, 134]       512
|    └─ReLU: 2-6                         [-1, 256, 134, 134]       --
|    └─ConvTranspose2d: 2-7              [-1, 128, 268, 268]       524,288
|    └─BatchNorm2d: 2-8                  [-1, 128, 268, 268]       256
|    └─ReLU: 2-9                         [-1, 128, 268, 268]       --
|    └─ConvTranspose2d: 2-10             [-1, 64, 536, 536]        131,072
|    └─BatchNorm2d: 2-11                 [-1, 64, 536, 536]        128
|    └─ReLU: 2-12                        [-1, 64, 536, 536]        --
|    └─ConvTranspose2d: 2-13             [-1, 3, 1072, 1072]       3,072
|    └─Tanh: 2-14                        [-1, 3, 1072, 1072]       --
==========================================================================================
Total params: 3,576,704
Trainable params: 3,576,704
Non-trainable params: 0
Total mult-adds (G): 120.18
==========================================================================================
```

Figure 1. The Generator architecture for the DCGAN model

Discriminator Architecture

The Discriminator architecture functions like a typical Convolutional Neural Network. The main goal of the Discriminator in the DCGAN architecture is to classify whether an input image is real or not (i.e Binary Classification). The Discriminator then outputs a probability that indicates how likely the input image is real which is then utilized to provide feedback to the Generator.

Layers

- **Input Layer:** The input to the Discriminator is a 64x64 image (*nc*x64x64) that is either from the training dataset or a fake image by the Generator.
- **Output Layer:** This layer contains a sigmoid function which is utilized to convert the predictions of the CNN into a probability.

```
==========================================================================================
Layer (type:depth-idx)                   Output Shape              Param #
==========================================================================================
├─Sequential: 1-1                        [-1, 1, 1, 1]             --
|    └─Conv2d: 2-1                       [-1, 64, 32, 32]          3,072
|    └─LeakyReLU: 2-2                    [-1, 64, 32, 32]          --
|    └─Conv2d: 2-3                       [-1, 128, 16, 16]         131,072
|    └─BatchNorm2d: 2-4                  [-1, 128, 16, 16]         256
|    └─LeakyReLU: 2-5                    [-1, 128, 16, 16]         --
|    └─Conv2d: 2-6                       [-1, 256, 8, 8]           524,288
|    └─BatchNorm2d: 2-7                  [-1, 256, 8, 8]           512
|    └─LeakyReLU: 2-8                    [-1, 256, 8, 8]           --
|    └─Conv2d: 2-9                       [-1, 512, 4, 4]           2,097,152
|    └─BatchNorm2d: 2-10                 [-1, 512, 4, 4]           1,024
|    └─LeakyReLU: 2-11                   [-1, 512, 4, 4]           --
|    └─Conv2d: 2-12                      [-1, 1, 1, 1]             8,192
|    └─Sigmoid: 2-13                     [-1, 1, 1, 1]             --
==========================================================================================
Total params: 2,765,568
Trainable params: 2,765,568
Non-trainable params: 0
Total mult-adds (M): 106.58
==========================================================================================
```

Figure 2. The Discriminator model architecture for DCGAN used for classifying real vs synthetically generated images

Loss Function:

$$\min_G \max_D \mathop{\mathbb{E}}_{\boldsymbol{x}\sim\mathbb{P}_r}[\log(D(\boldsymbol{x}))] + \mathop{\mathbb{E}}_{\tilde{\boldsymbol{x}}\sim\mathbb{P}_g}[\log(1 - D(\tilde{\boldsymbol{x}}))] \quad (1)$$

The DCGAN uses a minimax function between the Generator and the Discriminator. Each network is trying to maximize its objective; however maximizing one will cause the other one to minimize. For example, when you maximize the discriminator, it is trying to differentiate between the "real" and "fake". So when it is maximized, the generator won't be able to make images that can fool the discriminator, causing the Generator's loss function to move in the opposite direction. This same logic applies when we maximize the Generator. So through this minimax game, the DCGAN loss function tries to balance the Generator and Discriminator loss functions, which would help achieve a system where the Generator creates images that the Discriminator cannot differentiate from real images.

Note: The loss plots for the DCGAN models are added in the last page of this report.

2.2.2. WGAN-GP

The Wasserstein GAN is an alternative generative network that builds upon the DCGAN. The WGAN-GP maintains the same generator and discriminator relationship, but mainly differs in the GAN's loss calculation. The WGAN-GP's main objective is to minimize the Wasserstein-1 (Earth-Walker) distance, a rough measure of the distance to transform the latent distribution to the real distribution (Gulrajani et al., 2017). Additionally, a small gradient penalty (GP) is added to the loss function to ensure smooth training, hence the name WGAN-GP.

Generator Architecture:

The WGAN generator network is functionally and structurally very similar to the DCGAN's network. The only difference is the parameters and number of convolution blocks used (changed to appropriately fit the datasets) in the network.

Discriminator Architecture:

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
   ConvTranspose2d-1          [-1, 512, 67, 67]         819,200
       BatchNorm2d-2          [-1, 512, 67, 67]           1,024
              ReLU-3          [-1, 512, 67, 67]               0
   ConvTranspose2d-4        [-1, 256, 134, 134]       2,097,152
       BatchNorm2d-5        [-1, 256, 134, 134]             512
              ReLU-6        [-1, 256, 134, 134]               0
   ConvTranspose2d-7        [-1, 128, 268, 268]         524,288
       BatchNorm2d-8        [-1, 128, 268, 268]             256
              ReLU-9        [-1, 128, 268, 268]               0
  ConvTranspose2d-10         [-1, 64, 536, 536]         131,072
      BatchNorm2d-11         [-1, 64, 536, 536]             128
             ReLU-12         [-1, 64, 536, 536]               0
  ConvTranspose2d-13          [-1, 3, 1072, 1072]         3,075
             Tanh-14          [-1, 3, 1072, 1072]               0
================================================================
Total params: 3,576,707
Trainable params: 3,576,707
Non-trainable params: 0
----------------------------------------------------------------
```

Figure 3. The Generator model architecture for WGAN-GP. The architecture shown above is for the CIFAR-10 dataset. The exact architectures used for the other 2 datasets are similar in layers but differ in terms of input channels and output channels.

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
            Conv2d-1          [-1, 64, 32, 32]           3,072
         LeakyReLU-2          [-1, 64, 32, 32]               0
            Conv2d-3         [-1, 128, 16, 16]         131,072
    InstanceNorm2d-4         [-1, 128, 16, 16]               0
         LeakyReLU-5         [-1, 128, 16, 16]               0
            Conv2d-6           [-1, 256, 8, 8]         524,288
    InstanceNorm2d-7           [-1, 256, 8, 8]               0
         LeakyReLU-8           [-1, 256, 8, 8]               0
            Conv2d-9             [-1, 1, 5, 5]           4,097
================================================================
Total params: 662,529
Trainable params: 662,529
Non-trainable params: 0
----------------------------------------------------------------
```

Figure 4. The Discriminator model architecture for WGAN-GP. The architecture shown above is for the CIFAR-10 dataset. The exact architectures used for the other 2 datasets are similar in layers but differ in terms of input channels and output channels.

The WGAN-GP discriminator, also known as a critic since there is no classification, is also functionally and structurally very similar to the DCGAN's network but has a few critical differences.

Key Differences:

- **No Classification:** Unlike the DCGAN, the WGAN's discriminator doesn't classify whether an image is real or fake. Instead, the WGAN's discriminator provides a numerical value associated with the quality of the generated image (Gulrajani et al., 2017). Hence, the discriminator in the WGAN is also know as the critic.
- **No Batch Normalization:** Batch normalization maps a batch of inputs to a batch of outputs. However, the gradient penalty utilized in the WGAN-GP penalizes the norm of the discriminator's gradient with respect to each individual input (Gulrajani et al., 2017). Hence, utilizing batch normalization in WGAN-GP would nullify the use of gradient penalties.

Instead, the discriminator network employs instance normalization to normalize each sample individually.

- **No Sigmoid Activation in the Last Layer:** The Sigmoid activation is typically used to transform numerical values into a probability (i.e [0, 1]). However, the WGAN-GP discriminator is predicting the Wasserstein-1 (Earth-Mover) distance which can be any real number (Gulrajani et al., 2017). Using a Sigmoid activation would restrict the outputs to only [0, 1], destroying the Wasserstein-1 distance prediction.

Generator Loss Function:

$$L_{\text{generator}} = -\mathbb{E}_{\tilde{\mathbf{x}} \sim \mathbf{P}_g}[D(\tilde{\mathbf{x}})] \tag{2}$$

Equation (2) represents the loss function used to train the Generator. In simple terms, this expectation represents the average score the critic gives to fake images. The goal of the Generator is to maximize this score, but since we use gradient descent, we minimize the negative of this value which is conceptually the same as maximizing the expected score.

Discriminator Loss Function:

$$L_{critic} = \underbrace{\mathop{\mathbb{E}}_{\tilde{\boldsymbol{x}} \sim \mathbb{P}_g}[D(\tilde{\boldsymbol{x}})] - \mathop{\mathbb{E}}_{x \sim \mathbb{P}_r}[D(\boldsymbol{x})]}_{\text{Wasserstein Estimate}} + \underbrace{\lambda \mathop{\mathbb{E}}_{\hat{\boldsymbol{x}} \sim \mathbb{P}_{\hat{\boldsymbol{x}}}} \left[\left(\|\nabla_{\hat{\boldsymbol{x}}} D(\hat{\boldsymbol{x}})\|_2 - 1 \right)^2 \right]}_{\text{Gradient Penalty}} \tag{3}$$

Equation (3) represents the loss function used to train the Discriminator. There are 2 components involved in the loss calculation:

- **Wasserstein Estimate:** This component of the critic loss maximizes the distance between real and fake data, forcing the critic to assign higher values to real images and smaller values to fake (Gulrajani et al., 2017). By pushing the distributions farther apart, the critic is better able to estimate the Wasserstein-1 distance and therefore provide better gradient updates to the generator. Once again, since we are using gradient descent the order of calculation is flipped.
- **Gradient Penalty:** This component of the critic loss simply ensures that the calculated gradient is neither too small nor too large. In other words, this gradient penalty is added to ensure that the gradient is around 1 to ensure smooth training (Gulrajani et al., 2017).

Note: The loss plots for the DCGAN models are added in the last page of this report.

3. Experiments

3.1. Hyperparameter Tuning

3.1.1. DCGAN

For tuning the DCGAN hyperparameters, we utilized the Python library, Optuna, to find the best combination of hyperparameters for the Generator and Discriminator Networks of the GAN. Optuna is a hyperparameter optimization framework that is designed to automate the search for the best hyperparameter combination to optimize a specific metric. The actual search to find the best hyperparameter combination is done through a Bayesian approach, where it builds a probabilistic model of the function and uses the model to suggest the most promising hyperparameters to evaluate next (Balde, 2023). This is different from an exhaustive search or a random search because Optuna does not try every single combination possible, but a set number of trials. Also, in comparison to random search, it uses a probabilistic model instead of truly random to decide the next hyperparameter values to test (Balde, 2023). The hyperparameters I decided to tune for the DCGAN model were learning rate, latent vector size (z), feature maps in generator (*ngf*), feature maps in discriminator (*ndf*), and the $\beta 1$ (*beta1*) value for the Adam Optimizer.

- **Learning Rate, *lr*:** The Learning Rate is one of the most critical hyperparameters for training the model. If the learning rate is too high, it could lead to an unstable model, and a learning rate too low could result in slow convergence. The search space for Optuna is from a log-uniform distribution between 10^-5 and 10^-2.
  - Range of values from [1e-5, 1e-2]
- **Latent Size, *nz*:** The Latent Vector size defines the length of the input noise vector that goes into the Generator to create the synthetic image. A large latent vector could capture more detailed features, but also increases the training complexity.
  - Range of values from [50, 200]
- **Feature Maps in Generator, *ngf*:** The feature map size in the generator determines how many filters are used in the network's convolutional layers for the Generator Network, which could help incorporate a more complex structure in the image data
  - Discrete values from [32, 64, 128]
- **Feature Maps in Discriminator, *ndf*:** This is similar the feature maps in the Generator but for the Discriminator. The number of feature maps here can affect the model's ability to distinguish between real and fake.
  - Discrete values from [32, 64, 128]
- **$\beta 1$ Value for Adam Optimizer, *beta1*:** The Adam optimizer uses two beta parameters, beta1 and beta2, to control the moving averages of the gradient and its square. The Beta1 value controls the momentum term, which affects the stability of the optimization.
  - Range of values from [0.3, 0.9]

Table 1. Tuned Hyperparameter Values for DCGAN

HYPERPARAMETERS	MNIST	CIFAR-10	ANIME
LR	0.000776	0.0002	0.002
NZ	141	100	100
NGF	32	64	64
NDF	32	64	64
BETA1	0.427	0.5	0.3

3.1.2. WGAN-GP

The first step in the tuning process of the WGAN-GP was addressing the diverging loss of the Generator and Discriminator. Specifically, we initially observed that the Generator loss would approach $+\infty$ while the Discriminator loss would approach $-\infty$. This behavior rose from the Discriminator being able to perfectly differentiate real vs fake images in the first few iterations, rendering the subsequent gradient updates for the Generator useless. To overcome this issue, we first reduced the strength of the Discriminator by removing a convolution block and reducing the number of feature maps created in each layer. This change immediately stabilized the training. The remainder of the tuning for the WGAN was done manually (due to a lack of compute power) by experimenting with values for the following hyperparameters:

- **Discriminator Convolution Channels, *DCC*:** The Discriminator channels was an important parameter to tune because the feature maps generated in each layer of this network directly contributed to the overfitting of the Discriminator described above. Hence, finding the best in/out channel values for each layer was crucial in ensuring the network followed a healthy training pattern. The values for this parameter are represented in powers of 2, where each term represents the number of output channels in each layer of the Discriminator network (layer 1, 2, and 3; excluding the final layer).

- **Learning Rate, *LR*:** The learning rate was an important parameter to tune for the WGAN-GP to ensure that both the Generator and Discriminator were able to gather enough information to learn the features while also minimizing the convergence time.

- **β1, *BETA1*:** The Beta 1 value was an important parameter to tune for the WGAN-GP as the β1 values control how much of the previous gradient calculations are included in the running average. This parameter essentially helps control the smoothness of the gradient updates. The final values used for this parameter is adopted from (Gulrajani et al., 2017).

- **Training Epochs, *EPOCHS*:** The training epochs was an important parameter to tune for the WGAN-GP because it helped control the amount of training for the model. In other words, this parameter helped ensure that the Generator and Discriminator maintain a healthy relationship with neither of them overpowering the other.

- **Critic Loss Calculations per Iteration, *n_critic*:** The n_critic was crucial parameter to tune because it influences how many times the Discriminator loss is calculated in each iteration. By finding the optimal n_critic, the Discriminator maintained a good prediction of the Wasserstein-1 distance and provided meaningful gradient updates.

3.2. DCGAN Results

The DCGAN model showed promising results for each of the datasets. Based on visual comparison, we see that the Anime Face dataset performed the best, second was MNIST, and in last place was the CIFAR-10 dataset. The following takeaways summarize the performance and observations for each dataset on the DCGAN model.

Table 2. Tuned Hyperparameter Values for WGAN-GP

HYPERPARAMETERS	MNIST	CIFAR-10	ANIME
LR	0.01	0.0005	0.0005
DCC	$[2^4, 2^5, 2^6]$	$[2^6, 2^7, 2^8]$	$[2^6, 2^7, 2^8]$
BETA1	0.0	0.0	0.0
EPOCHS	32	64	64
N_CRITIC	7	5	5

3.2.1. MNIST DATASET

Figure 5. Real vs Generated Images for MNIST Dataset

Qualitative Observation: The generated images produce recognizable digits that achieve coherent outputs after several training epochs. Overall there is fine-details seen in the digits, however, some limitations were found in more complex circles or curves. These are seen in digits such as "6", "5", or "2", where we see more of a "blob" shape that is less identifiable.

Quantitative Observation: We did not incorporate quantitative measures for the MNIST dataset since it's a relatively simple dataset. We relied on qualitative measures to evaluate the quality of the generated images.

3.2.2. CIFAR-10 DATASET

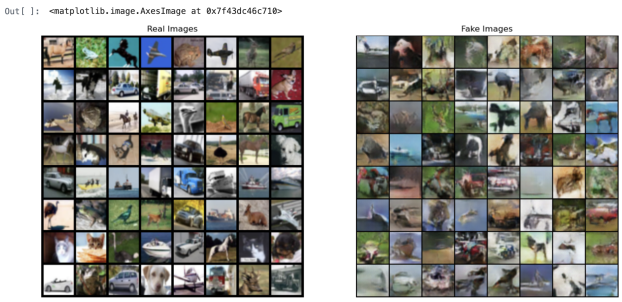

Figure 6. Real vs Generated Images for CIFAR-10 Dataset

Qualitative Observation: Based on the figure, you can see how DCGAN produced semi-recognizable images but also lacked fine

details for more complex categories like airplanes and animals. For example, the synthetically generated car images are shown in relatively greater detail. But, the rest of the classes lack this quality, where you can see the general shape of animals like a dog or a bird, but can't see the details. Overall, based on qualitative observations, the DCGAN performed the worst on the CIFAR-10 dataset as opposed to the MNIST or Anime Face datasets.

Quantitative Observation: We did not incorporate quantitative measures for the CIFAR-10 dataset due to the relatively small training and testing dataset sizes used which wouldn't have produced meaningful or accurate values. We relied on qualitative measures to evaluate the quality of the generated images.

3.2.3. Anime Face Dataset

Figure 7. Real vs Generated Images for Anime Face Dataset

Qualitative Observation: The generated faces mimicked the Anime art-style very well and were of decent-quality. The generated images also contained a good variety as observed through the hair color, facial expression, and eyes. However, we noticed that the DCGAN results contained noticeable deformities, especially mismatching eye colors or misplaced facial components, that we did not notice in the original images.

Quantitative Observation: For more quantitative metrics, we used the Fréchet Inception Distance (FID) and the Inception Score (IS) to tell us how well the model performed.

- **Fréchet Inception Distance (FID): 0.029362**
  - This tells us that there is a very high similarity between he generated anime faces and the real anime faces dataset. The FID score closer to 0 is more ideal and tells us the DCGAN model generated high-quality, realistically diverse anime faces.
- **Inception Score (IS): 1.896607**
  - This tells us that there is not as much variation between the faces, which we can visually see in all the faces since they all follow the same structure of the face and change minimal features.

3.3. WGAN-GP Results

Similar to the DCGAN, the WGAN-GP performed the best on the Anime Dataset, followed by the MNIST dataset, and lastly the CIFAR-10 dataset. The following sections summarize the performance and observations for each dataset on the WGAN-GP model.

3.3.1. MNIST Dataset

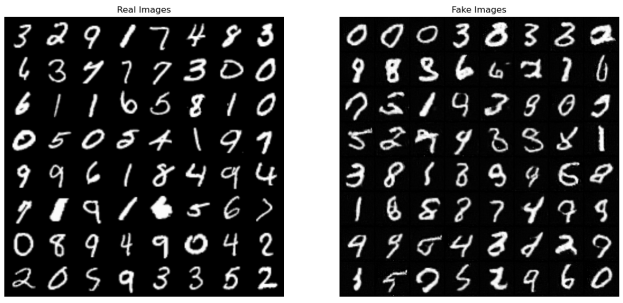

Figure 8. Real vs Generated Images for MNIST Dataset

Qualitative Observations: The generated images are of high quality and relatively easily recognizable. While some generated digits may seem like they are unfinished or of poor quality, it is important to note that even the original images contain blurry or unclear examples. This is an expected pattern since the MNIST dataset contains only handwritten numbers. However, there are still some issues with certain numbers like 8 and 3 likely due to insufficient training. The generated images also seem to have good variation with multiple different, recognizable images present for the numbers one through nine. Despite the irregularities, though, the WGAN-GP generated images are still of a higher quality with a lot fewer deformities compared to the DCGAN.

Quantitative Observations: We did not incorporate quantitative measures for the MNIST dataset since it's a relatively simple dataset. We relied on qualitative measures to evaluate the quality of the generated images.

3.3.2. CIFAR-10 Dataset

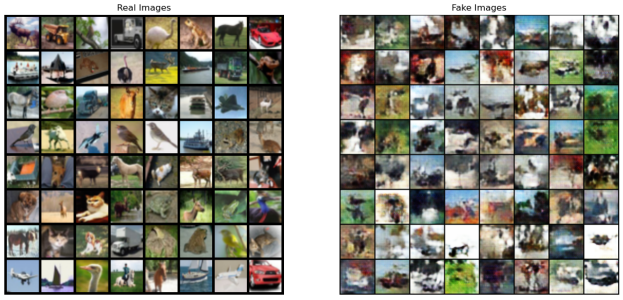

Figure 9. Real vs Generated Images for CIFAR-10 Dataset

Qualitative Observations: Like the DCGAN, the generated images for the CIFAR-10 dataset are really poor. Even though the generated images are blurry, this result is expected, since the real images are also blurry. The main issue with the generated images is the lack of detail. Although the fake images have enough variation to tell them apart from each other, they lack the structural coherence to clearly identify specific objects. A big contributor to this issue was probably the limited training time. Since the WGAN-GP was more computationally intensive than the DCGAN, the model was only trained for 70 epochs. Having a longer training

period would likely have helped the model capture fine detail in the generated images. For these reasons, the WGAN-GP performed worser on CIFAR-10 dataset than the DCGAN.

Quantitative Observations: We did not incorporate quantitative measures for the CIFAR-10 dataset due to the relatively small training and testing dataset sizes used which wouldn't have produced meaningful or accurate values. We relied on qualitative measures to evaluate the quality of the generated images.

3.3.3. Anime Face Dataset

Figure 10. Real vs Generated Images for Anime Face Dataset

Qualitative Observations: The WGAN-GP had great results for the Anime Face dataset, producing fake images that resemble the art style and detail of the real images better than the DCGAN. The generated outputs also displayed a diverse range of facial features, expressions, and hairstyles, highlighting the GAN's success in modeling the variation. However, one recurring trend we noticed was the unnatural rendering of eyes in certain anime figures, although this was much less prominent in these images compared to the DCGAN generated images. This distortion can likely be reduced with more training.

Quantitative Observations: For the Anime Face Dataset, we utilized the same two quantitative metrics to assess the quality of the outputs as for the DCGAN:

- **Fréchet Inception Distance (FID): 0.0239**
  - Since the FID score is really low, this indicates that the distribution of between the real images and fake images is very close. In other words, the FID score for the Anime Face dataset statistically demonstrates that the generated images have high image quality and realism. This quantitative measurement is consistent with the qualitative measurements along with the FID score of the DCGAN on this dataset observed above, reinforcing the superior performance of the WGAN-GP on this dataset compared to the DCGAN.
- **Inception Score (IS): 1.949**
  - An inception score of ~ 2 is very low. Given that the FID score is super low, the IS score is likely small due to a lack of variety in the outputs rather than the quality of the images. However, it is important to note that since this dataset mainly contains animated faces, the variation in the dataset is already limited. Hence, the low IS score might not be as meaningful, especially when we were visually able to observe the diversity in the generated images. The IS score for the WGAN-GP was also higher than the DCGAN, consistent with the observations from above.

3.4. Main Takeaway

The thorough experimentation of the 2 generative adversarial networks-DCGAN and WGAN-GP-revealed that the WGAN-GP outperformed the DCGAN in generating high-quality, diverse images. Specifically, the WGAN-GP produced better images on the MNIST and Anime Face datasets, while the DCGAN yielded better results on the CIFAR-10 dataset.

Table 3. Overall Head-to-Head Comparison

Dataset	Best Performing Model
Anime Face	WGAN-GP
MNIST	WGAN-GP
CIFAR-10	DCGAN

Note: The dataset column is ranked by intra-model performance (i.e each GAN had the best results, compared to the other datasets, for the Anime Face, followed by MNIST, and lastly CIFAR-10)

We also noticed that both GAN architectures exhibited their poorest performance on the CIFAR-10 dataset, particularly compared to their performance on the MNIST and Anime face datasets. Although the CIFAR-10 dataset only includes ten classes, the variability within each class was probably too great for either of the models to capture sufficient meaningful detail, resulting in the poor image quality we observed. In practice, this actually made the CIFAR-10 dataset the most challenging dataset for both GAN models. Despite the underwhelming results, the DCGAN outperformed the WGAN-GP on this task as it produced images with enough structural coherence to somewhat resemble the different classes of objects. In contrast, the outputs from the WGAN-GP were largely incoherent and contained mostly meaningless blobs, likely a direct result of decreased training time (only 70 epochs compared to 200 for the DCGAN) which was reduced to limit computational overhead.

Table 4. Summary of the Fréchet Inception Distance (FID) and Inception Score (IS) Comparison for the Anime Dataset

Model	FID	IS
DC-GAN	0.0294	1.897
WGAN-GP	0.0239	1.949

Note: A smaller FID score indicates better quality fake images (when compared to real images for the corresponding dataset) and a higher IS score indicates better quality (without comparison to the real images for the corresponding datasets) and variation in the images.

Overall, these results suggest that the DCGAN is better suited at capturing fine-detail in datasets with high intra-class variation, while the WGAN-GP excels in modeling more uniform datasets. However, despite each model's strengths, it is important to note

that the performance differences come at the expense of different computational overhead. Generally, the WGAN-GP was a lot more stable in training but required substantially more compute and training time. For instance, the WGAN-GP, despite producing better results on the MNIST dataset, was trained for a 100 epochs whereas the DCGAN produced reasonable images with only 5 epochs of training. In this sense, the DCGAN can be viewed as more computationally efficient. Nonetheless, our experiments deemed that the WGAN-GP produced better quality images on the MNIST and Anime Face datasets, while the DCGAN performed better on the CIFAR-10 dataset.

4. Conclusion

Generative modeling is a rapidly advancing area in machine learning, opening the door to a range of impactful use cases. In this study, we compared two well-known GAN architectures—DCGAN and WGAN-GP—on their ability to generate high-quality images on three datasets of increasing complexities: MNIST, CIFAR-10, and Anime Faces. After careful hyperparameter tuning and analysis, we concluded that the DCGAN excelled at modeling datasets with high intra-class variation, like the CIFAR-10 dataset, whereas the WGAN-GP consistently produced more realistic and visually coherent images on more balanced datasets like MNIST and Anime Faces. Interestingly, we also observed that both models exhibited the same overall trend in image quality across the datasets: Anime faces exemplified the best results, followed by the MNIST digits, and lastly the CIFAR-10 objects. While our results were promising given the limited time frame and computational resources, we believe that further training, architectural refinement, and hyperparameter tuning could significantly enhance image quality and diversity.

Future work can focus on incorporating more complex generative architectures such as StyleGAN, diffusion models, other attention-based models, or even hybrid GAN-transformer models for greater expressivity. Ultimately, as generative models continue to improve, they hold tremendous potential to revolutionize industries like health care, entertainment, design, and more, giving machines the capability to not only classify and predict but to imagine and create. Comparison studies like this provide critical insight in observing the strengths and weaknesses of different models.

References

Balde, B. Bayesian sorcery for hyperparameter optimization using optuna. *Medium*, 2023.

Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y. Generative adversarial networks, 2014. URL https://arxiv.org/abs/1406.2661.

Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., and Courville, A. Improved training of wasserstein gans, 2017. URL https://arxiv.org/abs/1704.00028.

Radford, A., Metz, L., and Chintala, S. Unsupervised representation learning with deep convolutional generative adversarial networks, 2016. URL https://arxiv.org/abs/1511.06434.

Note: The next pages contain the loss plots and side-by-side comparisons of the generated outputs of the DCGAN and WGAN-GP on the different datasets.

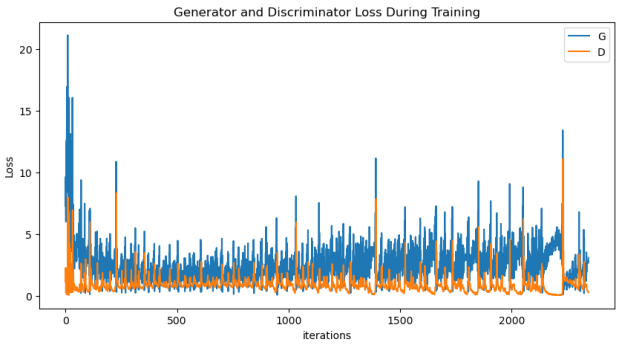

Figure 11. MNIST Dataset Loss Curve for the DCGAN

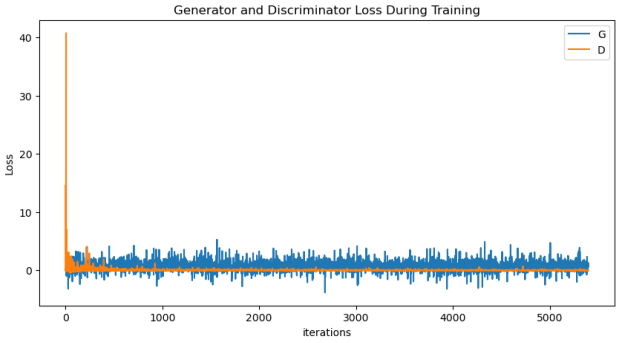

Figure 14. MNIST Dataset Loss Curve for the WGAN-GP

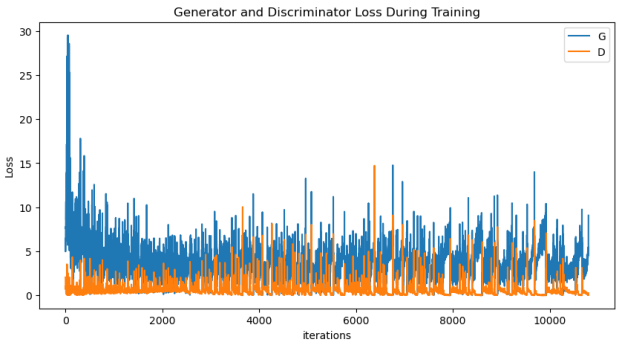

Figure 12. CIFAR-10 Dataset Loss Curve for the DCGAN

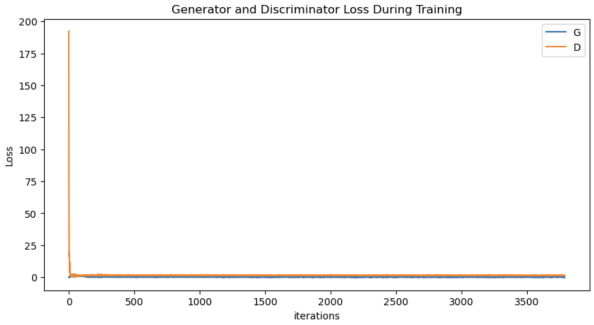

Figure 15. CIFAR-10 Dataset Loss Curve for the WGAN-GP; The loss curves are hard to see here, but they generally maintain the same trend observed in the loss plots for MNIST (above) and Anime Faces (below)

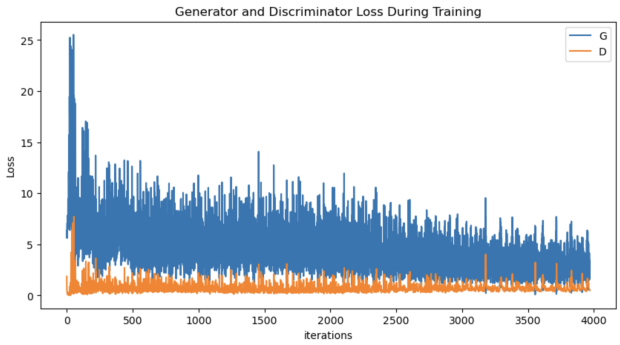

Figure 13. Anime Face Dataset Loss Curve for the DCGAN

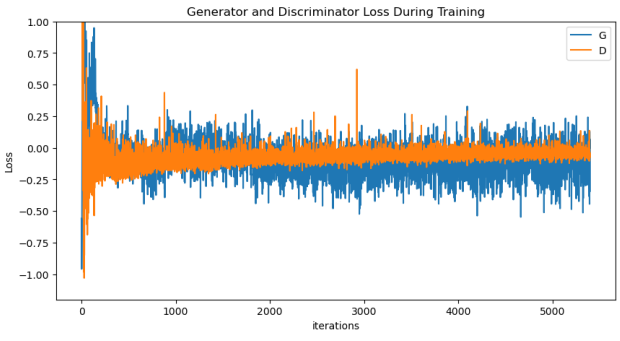

Figure 16. Anime Face Dataset Loss Curve for the WGAN-GP

Figure 17. Fake MNIST digits Generated by DCGAN

Figure 19. Fake MNIST digits Generated by WGAN-GP

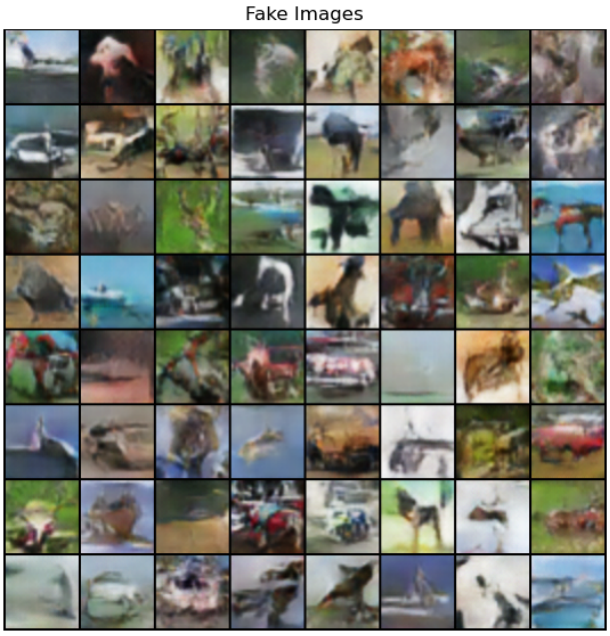

Figure 18. Fake CIFAR-10 Objects Generated by DCGAN

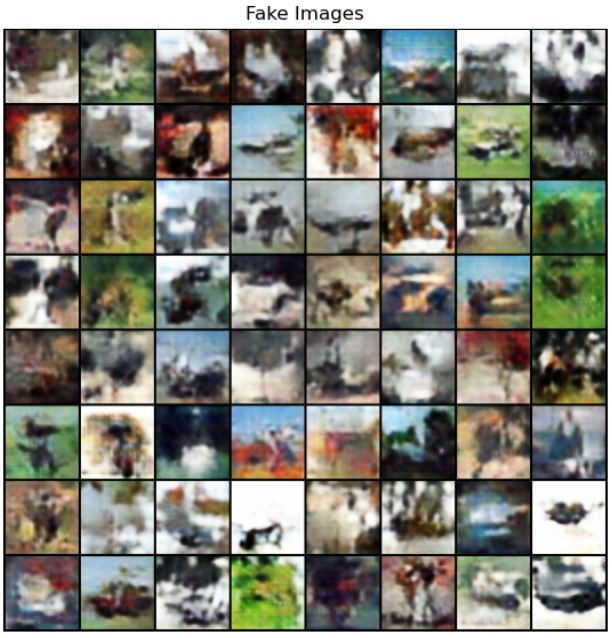

Figure 20. Fake CIFAR-10 Objects Generated by WGAN-GP

Figure 21. Fake Anime Faces Generated by DCGAN

Figure 22. Fake Anime Faces Generated by WGAN-GP